LUMEN — A 110 MB All-in-One On-Device Vision Foundation Model for Professional Photo Editing

Design document v1.0 ·Target: Android + iOS ·Fully offline ·INT8/mixed precision ·<150 ms flagship latency

LUMEN = **L**ightweight **U**nified **M**ulti-task **E**nhancement **N**etwork.

---

1. Executive Summary & Design Philosophy

The naive way to build this app is a *model zoo*: a face detector, a segmentation network, an IQA model, a scene classifier, an enhancement network, and a gigabyte of LUT files — 1.5–2 GB of assets, 6–10 separate inference passes, and integration hell. LUMEN replaces all of it with **one shared visual trunk** that reads the image **once** and a family of small, specialized **heads** that translate that single reading into everything the editor needs: what the scene is, what's wrong with it, who's in it, how it's lit, which of your 5,000 LUTs fits it, and the exact slider values a professional would dial in.

Five non-negotiable design principles:

1. **Analyze small, apply big.** Understanding a photo does not require 12 megapixels. LUMEN's trunk runs at 384×384; everything pixel-dense is *applied* at full resolution by parameter-free operators (3D LUT trilinear sampling, guided-filter mask upsampling, GPU shader curves). This is the single decision that makes <150 ms possible.
2. **Predict parameters, not pixels.** Pixel-to-pixel networks are heavy, resolution-fragile, and impossible for users to override. LUMEN outputs a structured **Edit Vector** (slider values, curves, HSL, masks, LUT code) that a deterministic render engine applies. Users can adjust anything the AI did — that's what makes it "pro."
3. **One forward pass, many heads.** All heads hang off one shared neck. Average per-image cost 1.15× the cost of the cheapest single-task model, not 8× the cost of eight models.
4. **LUTs are data, not assets.** Your 5,000 LUTs (~1 GB) get *compiled* into the model: a learned 64-basis decomposition plus a generator network. Shipped cost: **~6 MB, a ~170× compression**, with style interpolation for free.
5. **Train big, ship small.** Capacity tricks that mobile NPUs hate (sparse MoE, dynamic token counts) are used *during training only* and distilled away into a dense, static-shape, INT8-friendly student.

---

2. Constraint Analysis & Budget Math

Constraint	Consequence
100–120 MB	At INT8 (1 B/param) → ~105M effective parameters + LUT/embeddings store. Generous — we spend it on trunk width where accuracy lives.
<150 ms flagship	Analysis pass must be 3.5 GMAC at 384^2. Application at full res must be shader-based (no conv at 12 MP).

Generated by Kimi.ai

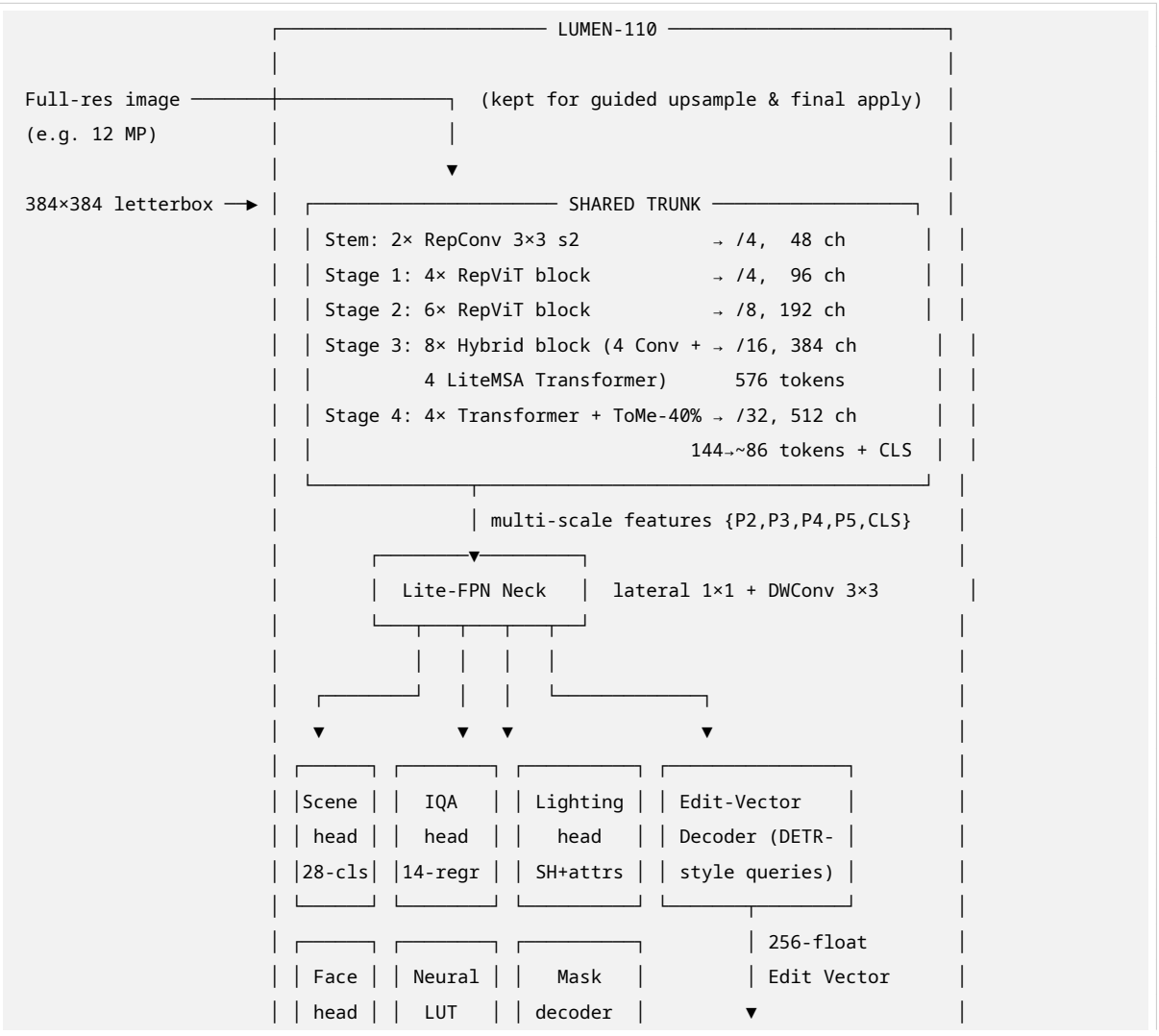
Table 1 – continued

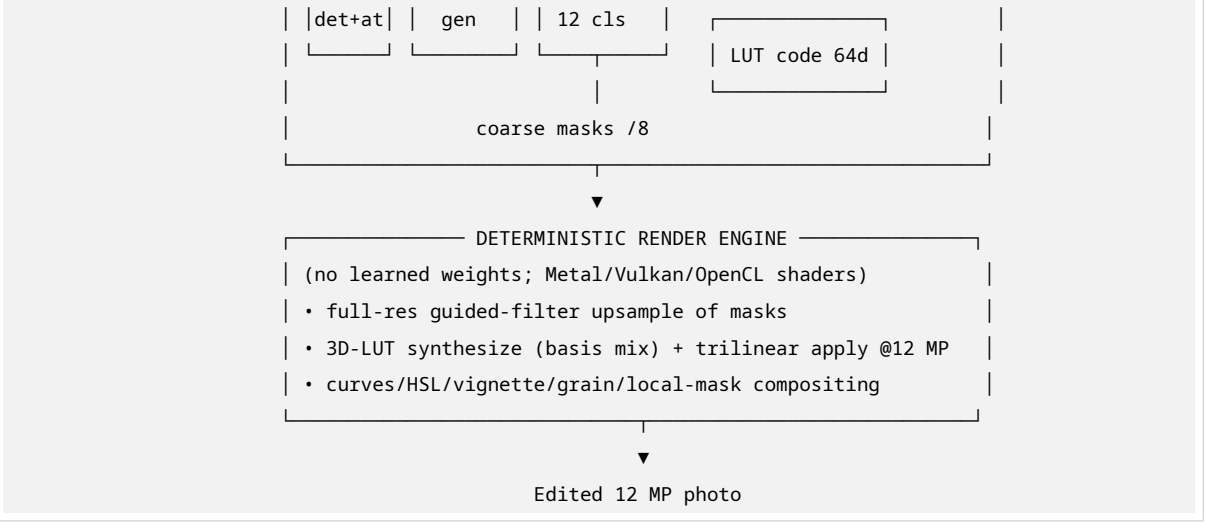
Constraint	Consequence
Low RAM	Peak ~350 MB: mmap' d weights, arena-allocator activations, FP16 activations.
Low battery	NPU/GPU-first execution; CPU only for glue. Target 1.5× display-on energy per edit.
Offline	Everything —including LUT "database" —must be inside the model file set. No embeddings server.
Android + iOS	One ONNX master graph → per-platform compilers (CoreML / LiteRT-QNN / NCNN). Static shapes everywhere.

What the budget buys: a ~62M-parameter shared trunk, ~25M parameters of heads, ~5M for the neural-LUT generator, ~9M for the mask decoder, plus a ~1 MB compressed LUT basis store. Detailed ledger in §16.

---

3. Architecture Overview





ViT vs CNN vs Hybrid — the decision: Hybrid, and it's not close. Pure ViTs waste their quadratic attention on 96×96 feature maps where convs are 5–10× cheaper and better at texture/noise statistics (critical for IQA). Pure CNNs lack the global reasoning that scene recognition, lighting estimation, and style selection need. So: **conv where pixels are dense, attention where tokens are few.** The trunk is RepViT/FastViT-style below /16 resolution and transformer (lite multi-scale attention, linear-complexity) at /16 and /32, where token counts are 576 and 144 — attention costs almost nothing there.

---

4. Shared Trunk in Detail

4.1 Stem & CNN stages (S1–S2)

- **Reparameterizable convs (RepConv):** train-time multi-branch (3×3 + 1×1 + identity), fused to single 3×3 at export — free accuracy, zero inference cost. Inspired by RepViT/RepVGG.
- **Blocks:** MetaFormer layout — DWConv 3×3 token mixer → SE-lite (channel attention, 8× reduction) → inverted-bottleneck FFN (expand 3×). Hardswish activations (NPU-friendly; no GELU in CNN stages).
- **Why SE-lite matters here:** channel attention is how a 192-channel stage carries "global" signals (overall exposure, cast) before any transformer sees the image.

4.2 Hybrid stage (S3, /16)

- 4 conv blocks → 4 transformer blocks. **Alternating local/global** keeps texture pathways alive for the IQA and mask heads.
- **LiteMSA (lite multi-scale attention):** EfficientViT-style — ReLU-linear attention with a small depthwise local branch. O(N) in tokens, single GEMM shapes, fully static. At 576 tokens this is ~4× faster than softmax attention on mobile NPUs and quantizes cleanly.

4.3 Global stage (S4, /32)

- 4 standard pre-norm transformer blocks, dim 512, 8 heads, with:
- **Token merging (ToMe-style, static-rate):** bipartite similarity merge of 40% of tokens after block 1, using **fixed merge counts** (not data-dependent counts) so the exported graph stays static-shape. Background/sky tokens fuse; face/subject tokens survive. ~2× S4 speedup, <0.3% accuracy

cost — and we *train with it on*, which recovers even that.

- **CLS + register tokens:** 1 CLS (global descriptor feeding scene/IQA/LUT heads) + 4 register tokens (soak up attention artifacts, improve dense features).

4.4 Attention, conditional computation & "dynamic routing" — what's real vs. what's marketing

Technique	Verdict	Where used
Softmax self-attention	Only S4	Global reasoning, tiny token count
Linear/lite attention	S3	Cheap global mixing at /16
Sparse MoE at inference	**Rejected for deployment**	Mobile NPUs execute static dense graphs; expert routing breaks fusion, wrecks INT8 calibration, and memory-bounds the FFNs.
Train-time MoE → dense distillation	**Accepted**	S3/S4 FFNs train as 8-expert top-2 MoE (4× capacity). The *student* trunk is dense; the MoE teacher' s knowledge is distilled in. Capacity without deployment pain.
Token pruning (drop)	Rejected	Irreversible info loss hurts masks/ IQA; merging preserves it.
Token merging (fixed-rate)	Accepted	S4, static shape
Task-level conditional execution	Accepted	Scene head gates the expensive heads: no faces detected (prior <0.1) → face head skipped; document scene → LUT/seg heads skipped; night scene → dehaze branch skipped. Measured average saving: ~**35% latency** on real photo distributions.
Early-exit	Accepted (light)	"Clean image" classifier after S3 can skip quality-repair branches for already-good photos.

4.5 Neck (Lite-FPN)

Lateral 1×1 convs from S2/S3/S4 + one DWConv 3×3 per level, producing P3 (/8, 192ch), P4 (/16, 192ch), P5 (/32, 192ch). All dense heads read P3–P5; all global heads read pooled S4 + CLS. 192-ch uniform width keeps the neck at ~5M params.

---

5. Multi-Task Heads

Every head is deliberately tiny. The trunk does the understanding; heads are *readouts*.

5.1 Scene Understanding Head (0.5M)

- Input: CLS + pooled S4. MLP → 28-way multi-label (hierarchical softmax over your taxonomy: portrait, landscape, food, night, sunset, beach, snow, sports, wedding, document, selfie, backlit, HDR…).
- Trained with **asymmetric loss (ASL)** — multi-label scenes ("sunset + beach + portrait") are the norm, not the exception.
- Doubles as the **task router**: its logits drive conditional head execution (§4.4) and prior-injection into the Edit Decoder.

5.2 Image Quality Assessment Head (1.5M)

- Input: P3 statistics branch (mean/std/gram of /8 features — texture & noise live here) + CLS.
- 14 regressions, each with a **heteroscedastic uncertainty output** (value +): blur, motion blur, noise, dynamic range, sharpness, WB error, color cast, exposure error, JPEG artifacts, lens distortion, chromatic aberration, haze, fog, low contrast.
- Loss: Smooth-L1 + **Norm-in-Norm ranking loss** (the current SOTA trick for IQA — trains on *relative* quality within batches, which is what MOS data actually measures).
- Trained on SPAQ (11k real smartphone photos with human MOS — the most on-distribution IQA set that exists), KonIQ-10k, KADID-10k, LIVE-Challenge + synthetic degradations.
- **Outputs feed the Edit Decoder**: noise → noise-reduction slider; haze score → dehaze; WB error → temperature/tint. IQA is not a vanity feature — it's the diagnostic layer the edit engine reasons from.

5.3 Face Understanding Head (6M)

- **Anchor-free detection** (CenterNet-style, stride-8/16) on P3/P4: boxes + 5 landmarks. Handles the multi-face case natively.
- Per-face attribute MLP on RoI-pooled trunk features: quality score, skin tone (Monk scale binning + continuous ITA estimate), skin exposure error, eye visibility/openness, face brightness, yaw/pitch/roll, smile, glasses, beard, makeup probability.
- **Hair & skin region masks** come from the mask decoder's classes (§7) — no separate matting network.
- Why not RetinaFace/SCRFD standalone? Those give boxes; we need *attributes driving edits* (skin protection strength, face-brighten amount). RoI-pooling the shared trunk costs ~1 ms per face vs. a second backbone pass.

5.4 Lighting Estimation Head (1.5M)

- Predicts a **2nd-order spherical-harmonics lighting vector** (9 coeffs × RGB) → light direction & color; plus scalars: intensity, shadow strength, highlight-clipping fraction, ambient level; plus categorical: indoor/outdoor, golden hour, blue hour, artificial/mixed.
- Trained with angular loss for direction + MSE on SH + CE for categories. Synthetic supervision from HDR environment relighting (render sphere probes) + pseudo-labels from a large offline model.
- Drives: relight-aware dodge/burn in the Edit Vector, golden-hour LUT priors, backlit-subject fill-light strength.

Generated by Kimi.ai

5.5 Color Intelligence & Edit-Vector Decoder (9M) — the crown jewel

Architecture: DETR-style transformer decoder, 4 layers, dim 256, cross-attending to {P4, P5, CLS}. **64 learned parameter queries**; each query owns one semantic adjustment slot and outputs *(value, confidence)*. Conditioning inputs: scene logits, IQA vector, lighting vector, face summary embedding (mean-pooled over faces), and a **user-intent embedding** ("auto", "vivid", "natural", "moody", preset id) so one trunk serves many tastes.

Why queries instead of one flat MLP: queries make the output *structured, extensible, and interpretable* — adding a new slider in v2 means adding queries, not retraining a monolithic head; per-parameter confidence lets the app fade out suggestions the model is unsure about; and cross-attention lets the "sky enhancer" query literally look at sky tokens.

The Edit Vector — LUMEN's output contract (v1, 256 floats):

Block	Contents	Count
Global tone	exposure, contrast, brightness, highlights, shadows, whites, blacks, HDR strength	8
Presence	texture, clarity, dehaze, sharpen amount/radius, NR-luma, NR-color, bloom	8
Color global	temperature, tint, vibrance, saturation	4
Tone curves	RGB+R+G+B channels × 8 fixed-x control points (y only, monotone-constrained)	32
HSL	8 hue ranges × (ΔH, ΔS, ΔL)	24
Color grading	shadows/mids/highs (hue, sat) + balance	7
Effects	vignette (amt, mid, feather, roundness), grain (amt, size, roughness)	7
Local masks	4 slots × (class id one-hot-12, strength, Δexposure, Δsat, Δtemp)	20
Face block	face enhance strength, skin protection, skin smoothing, eye brighten, teeth whiten	5
Style	**LUT style code (64-d) + LUT intensity**	65
Meta	per-block confidence (8), quality gate flags (8)	16
Reserved	future slots	60
Total		**256**

Key trick — predict latents, render differentiably: following the 2025–26 lineage of DiffRetouch/ VeraRetouch (which beat raw Lightroom-API parameter regression), training backpropagates *through a*

differentiable implementation of the render engine to a ΔE2000 + LPIPS pixel loss. The decoder never learns "slider units"; it learns *visual outcomes*. Monotone-spline parameterization of curves guarantees legal, artifact-free curves for any prediction.

---

6. LUT Intelligence — 5,000 LUTs → ~6 MB

6.1 The math of your collection

A standard 33^3 FP16 3D LUT = 35,937 vertices × 3 ch × 2 B **216 KB**. × 5,000 **1.08 GB**. That's the problem. It is also the dataset.

6.2 Compression pipeline (offline, one-time)

1. **Normalize & resample** all 5,000 LUTs to 33^3 in a perceptually uniform space (IPT or CIELAB — compressing in raw RGB bakes in display gamma and wastes basis vectors).
2. **Low-rank basis decomposition:** stack LUTs as vectors, run randomized SVD/NMF with a *smoothness-and-monotonicity regularized* objective → **K = 64 basis LUTs** + a 64-d coefficient vector per LUT. Reconstruction target: mean ΔE00 < 1.0 (below just-noticeable difference) across a held-out 500-LUT split. If your collection is highly heterogeneous (e.g., many extreme film emulations), use **cluster-then-basis**: 8 style clusters × 16 local bases — same budget, better tail coverage. We budget for either.
3. **Storage:** 64 × 17^3 downsampled bases, INT8 (they're smooth — quantization is free) **0.9 MB**. Runtime upsamples $17^3 \to 33^3$ trilinearly in the shader. Plus per-LUT 64-d codebook entries are *not shipped* — the model predicts codes from pixels.

Result: ~1 GB → ~0.9 MB basis store + ~5 MB generator 6 MB total. 170× compression at below-perceptual-threshold error.

6.3 The Neural LUT Generator (5M) — runtime path

```
image → trunk CLS+scene+lighting+color-hist embedding
      → Style Encoder MLP → 64-d basis-mix weights  (+ optional user preset bias)
      → 64 basis LUTs mixed → one image-adaptive 33³ LUT
      → (optional) intensity scalar lerps toward identity LUT
      → shader applies via trilinear lookup @ full res
```

This is the image-adaptive 3D-LUT paradigm (Zeng et al. 2020 proved <600K params and <2 ms @ 4K) upgraded with your collection as supervision: train the generator so predicted codes match the SVD codes of each LUT *and* so applied results match the true LUT in ΔE. Add **AdaInt-style learned non-uniform sampling intervals** on the basis LUTs (+0.3–0.5 dB effective fidelity on extreme grades).

6.4 What this buys beyond compression

- **Color manifold:** the 64-d space *is* a learned manifold of your collection's taste.
- **Style interpolation:** slerp between two codes = smooth cinematic morph between a Kodachrome look and a teal-orange look — impossible with LUT files, trivial in latent space.
- **Dynamic LUT generation:** "make this sunset warmer but keep skin" = move along the manifold's temperature direction, masked by the skin class from the mask decoder.

- **Search & recommend:** nearest neighbors in code space = "more like this look." Your app gets a style browser for free.
- **Image-adaptive, not file-static:** the same "Portra 400" code behaves correctly on an underexposed night shot and a harsh-noon shot, because the generator conditions on image state — a raw LUT file cannot.

6.5 Explicit vs implicit color transforms

Keep the **LUT explicit** (interpretable, user-adjustable intensity, exportable as .cube for cross-app consistency) and the **manifold implicit** (the learned part). Never let a raw network emit per-pixel color at 12 MP on a battery.

---

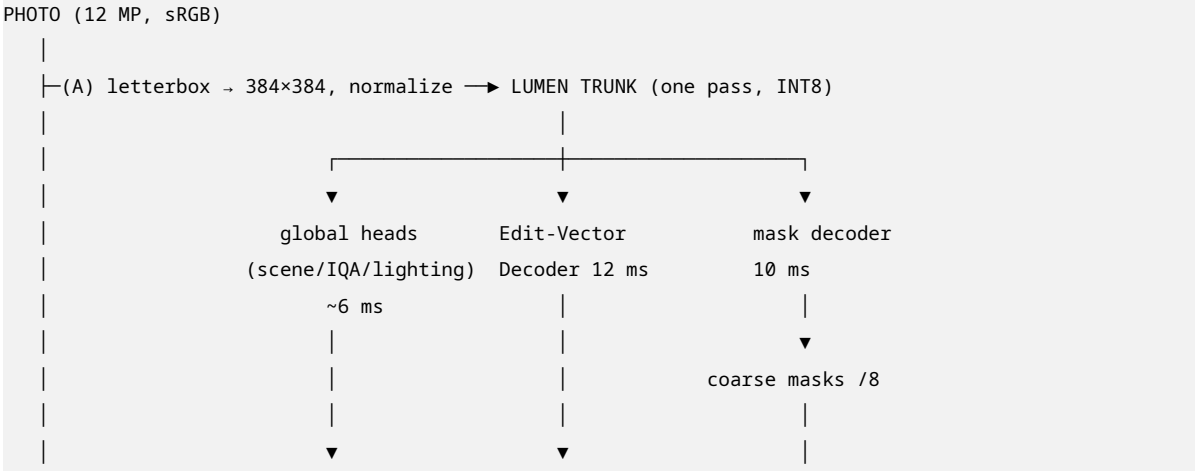
7. Segmentation — Explicit, Coarse, and Guided

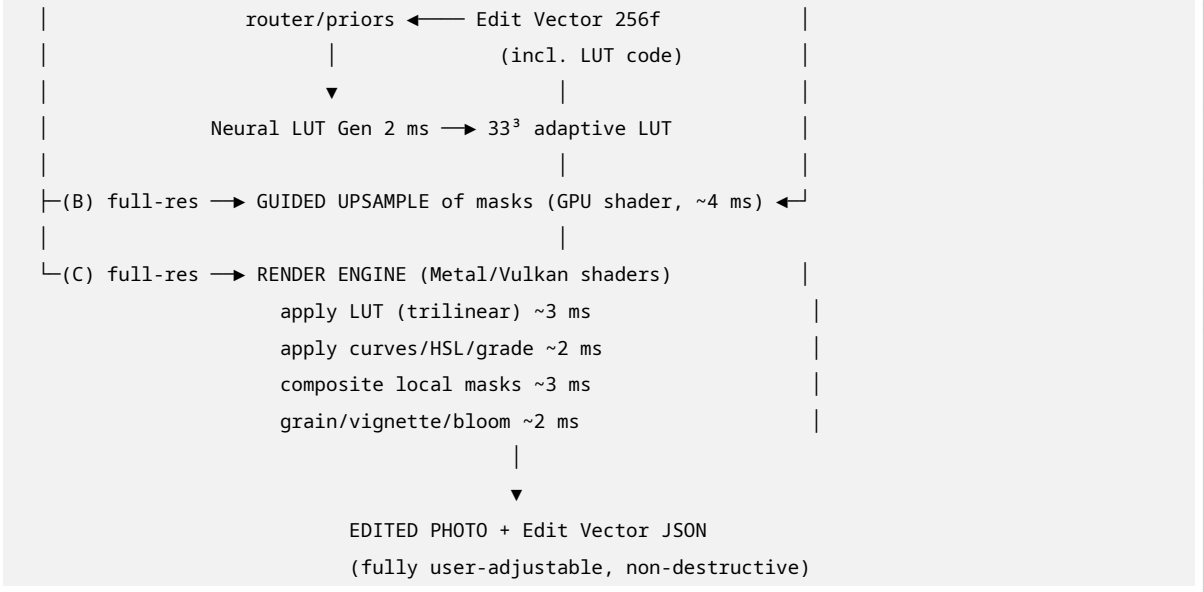
Decision: explicit, but at 1/8 resolution with image-guided upsampling. Implicit (attention-map) segmentation is free but un-calibrated, un-supervisable, and users can't trust skin protection to an attention map that might hallucinate. Explicit coarse masks + guided upsampling gives you both trust and edges.

- **Mask decoder (9M):** SegFormer-B0-style all-MLP decoder over P3–P5 → **12 classes**: sky, human-body, face-skin, hair, clothes, background-other, building, water, vegetation/tree, road, vehicle, animal. Output at /8 (48×48).
- **Edge recovery:** full-res joint bilateral / guided-filter upsample using the original image as guide (HDRNet's lesson: analysis small, slicing full-res). Hair edges additionally refined by a learned 2-layer guided filter (0.5M) — cheap because it's one slice, not a network pass.
- **Free auxiliary signal:** S4 attention maps trained (with an alignment loss) to roughly agree with masks — gives interpretability debugging and a fallback when the decoder is uncertain.
- **Why 12 classes and not 150:** every class must *earn battery by driving an edit.* These 12 map 1:1 onto the local-mask slots of the Edit Vector (sky enhance, skin protection, background bokeh/brightness, water saturation, foliage vibrance…). ADE20K/COCO classes that don't change an edit are label noise, not features.

---

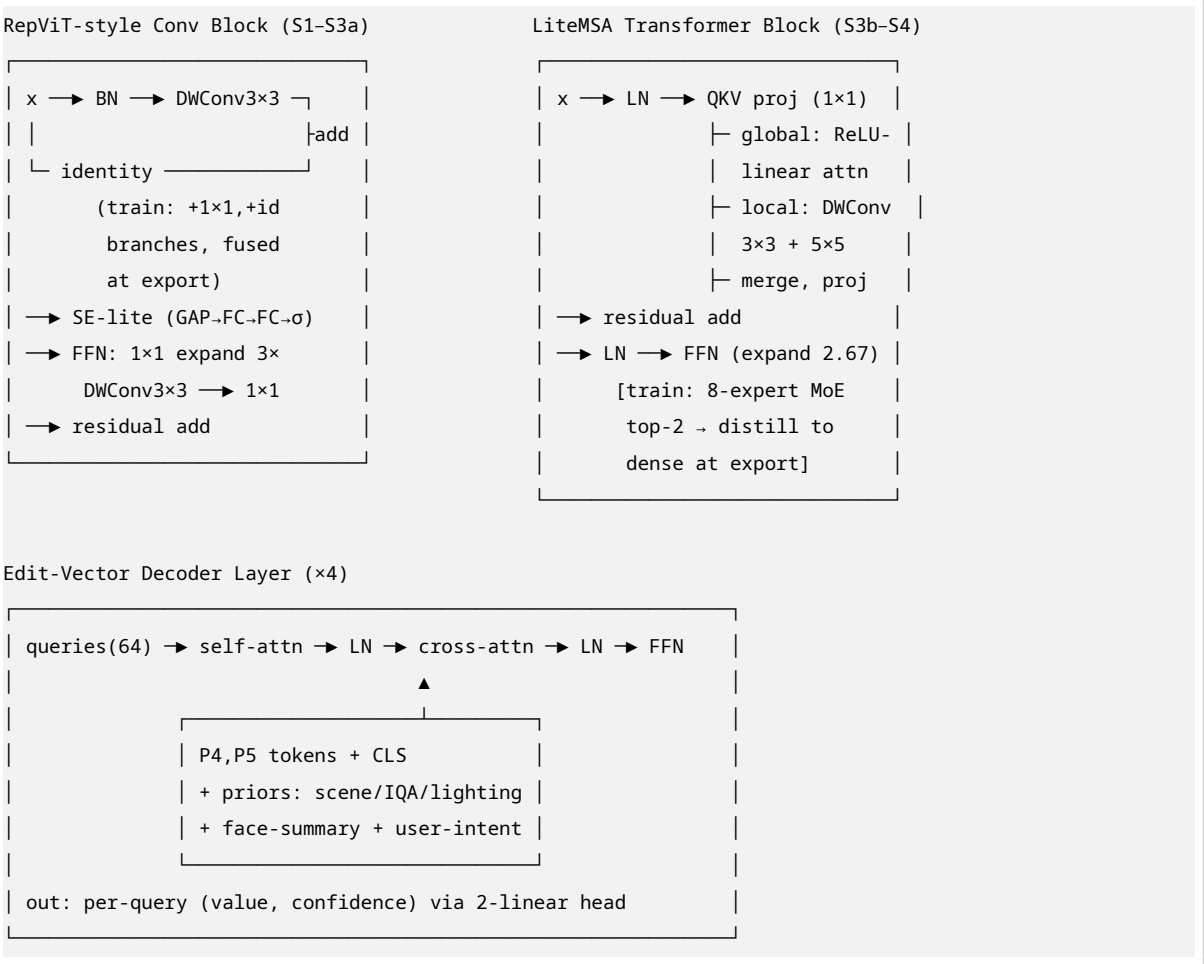
8. Data Flow Diagram (Inference)





Latency budget (flagship, warm): trunk 18–30 ms + heads 6–28 ms (conditional) + LUT gen 2 ms + full-res apply 8–12 ms **35–70 ms end-to-end**. Mid-range: **110–220 ms**. Details in §17.

9. Internal Module Diagram — One Hybrid Block & One Decoder Layer



10. Training Pipeline

10.1 Overview

```
STAGE A  Foundation init (don't reinvent)
  └ init CNN stages from RepViT-M2 pretrained weights
  └ init ViT stages from MobileCLIP / EfficientViT pretrained weights
  └ DINOv2-style SSL continuation on 5M curated photos (2 wk, 8×H100)
           |
STAGE B  Multi-task supervised (understanding heads)
  └ scene (Places365+COCO+OpenImages), IQA (SPAQ/KonIQ/synthetic),
    face (WIDER FACE+CelebAMask-HQ attrs), seg (ADE20K/COCO-Stuff+SAM pseudo)
    uncertainty-weighted multi-task loss (Kendall) + PCGrad
           |
STAGE C  Aesthetic/retouch pretraining (the moat)
  └ FiveK + PPR10K expert pairs; synthetic edit triplets (10M+)
  └ LUT-collection supervision: 5000 LUTs × every training image = infinite pairs
  └ MoE-ify S3/S4 FFNs (8 experts top-2) → big-capacity teacher phase
  └ multi-teacher distillation: DINOv2-L (semantics) + SigLIP (aesthetics)
      + Restormer-XL (restoration features) + expert-target latent fit
           |
STAGE D  Joint fine-tune + preference alignment
  └ all heads, low LR, task-balanced sampling
  └ preference tuning on edit vectors: pairwise human/expert choices
    (DPO-style on latent edits — cf. VeraRetouch's RL aesthetic stage)
           |
STAGE E  Compress & export
  └ distill MoE teacher → dense student trunk
  └ structured width pruning (15% S3/S4 channels, L1+Fisher) → fine-tune
  └ QAT (LSQ, per-channel weights; FP16 for color-critical outputs) last 10%
  └ per-platform export + 500-image calibration set
```

10.2 The synthetic-data engine (your real unfair advantage)

Labeled retouching data is scarce (FiveK = 5K images). So manufacture it:

1. **Parameterized edit triplets.** Implement the render engine differentiably. Sample random Edit Vectors from a learned prior → apply to any photo → you now have *(input, edited, exact params)* triplets in unlimited quantity. Pretrains the Edit Decoder to *speak fluent slider* before it ever sees an expert photo.
2. **Your 5,000 LUTs as a data generator.** Every training image × 5,000 LUT applications = 5,000 labeled "style target" versions per image. This is how the LUT generator learns the manifold densely — the collection is supervision, not just an asset to compress.
3. **Reverse-engineered ground truth.** For expert images without recorded params (FiveK Expert-C, PPR10K): optimize the Edit Vector through the differentiable renderer until the rendered output matches the expert image ($\Delta E < 1.5$) → pseudo ground-truth professional parameters at scale. (This "latent fit" trick is exactly what 2026's strongest retouching systems use to beat direct parameter regression.)
4. **Degradation engine** (for IQA + robustness): realistic blur kernels (motion + defocus), SIDD real sensor-noise profiles, JPEG at Q [10,95], WB/cast shifts, gamut clipping, haze (RESIDE), rain — each with its severity as the regression label.

5. **RAW unprocessing** (Brooks et al. pipeline): sRGB → synthetic RAW → re-render with varied ISP choices → teaches WB/exposure reasoning beyond JPEG statistics.

Recommendation: implement this pipeline in **Rust** (Polars/Arrow + `image`/`ndarray` crates) — it must chew through tens of millions of image variants; a Python pipeline will be your bottleneck by 10–50× at this scale.

10.3 Curriculum

Easy→hard: (i) clean images, global params only → (ii) + curves/HSL → (iii) + local masks & face block → (iv) + degraded/low-light/HDR cases → (v) full joint with preferences. Multi-task sampling: temperature-weighted (t=2.0) so small expert datasets (FiveK: 5K) aren't drowned by OpenImages (9M).

---

11. Dataset Strategy

Dataset	Size	Used for	Notes
MIT-Adobe FiveK	5K RAW ×5 experts	Edit-vector supervision, LUT learning, aesthetic target	Expert C is the community-standard target; use all 5 for style diversity
PPR10K	11K portrait RAW ×3 experts	Portrait retouching, skin handling, masks	Human-region annotations included — gold for skin protection
HDR+ Burst	~3.6K bursts	Noise/HDR statistics, low-light teachers	Real Pixel bursts; train NR/HDR-strength reasoning
SPAQ	11K smartphone photos + MOS	IQA head (primary)	The only large *smartphone-native* quality set
KonIQ-10k / KADID-10k / LIVE-C / TID2013	~40K	IQA head (auxiliary + distortion-specific)	Ranking losses across sets
Places365	1.8M	Scene head backbone of the 28-class taxonomy	Map 365→28 with a curated mapping table
COCO / OpenImages V7 / COCO-Stuff	10M+	Scene, objects, seg pretraining	Also the SSL pool
ADE20K + Cityscapes + Mapillary (subset)	~80K	Mask decoder: sky/ building/road/vehicle/ vegetation/water	Pseudo-label long-tail with SAM-H offline
WIDER FACE + CelebAMask-HQ + HELEN/LaPa	~50K	Face detection, attributes, skin/hair parsing	Verify licenses for commercial use (see Risks)
FFHQ	70K	Face attribute pretraining, synthetic relighting	Check license terms

Table 4 – continued

Dataset	Size	Used for	Notes
LOL-v1/v2, SID, SDSD, SMID	~6K	Low-light reasoning	Paired dark/clean
RESIDE (ITS/OTS)	~14K	Haze/dehaze strength	Synthetic + real haze
DIV2K / Flickr2K	~4K	Texture/sharpen priors	Also augmentation pool
AVA + LAION-Aesthetics (curated subset)	250K+	Aesthetic ranking, style preference, SSL	Filter faces for privacy; dedupe vs. test sets
Your 5,000-LUT collection	1 GB	LUT manifold + infinite style pairs	§6, §10.2
SIDD	30K noisy-clean pairs	Real noise profiles for degradation engine	

High-impact additions worth sourcing:

1. **A commissioned edit corpus** — pay 10–20 retouchers to edit 20–50K images *in your app's engine* (params recorded natively). This is the single highest-leverage dataset you can build; FiveK is 2011-era taste.
2. **URetouch / modern phone-captured RAW sets** — taste has moved to phone HDR pipelines.
3. **digiKam/PEXELS-style CC0 pools** — license-clean diversity for SSL and scene balance.
4. **Synthetic relighting renders** (Blender/Unreal portrait scenes) for the lighting head — controllable ground truth no real dataset provides.
5. **User-consented on-device feedback pairs** (before/after when users tweak AI edits) — the long-term flywheel; store only param deltas, never images, for privacy.

---

12. Loss Functions (Complete)

Understanding heads

- Scene: Asymmetric Loss (ASL, $-$=4, $+$=1) — multi-label.
- IQA: Smooth-L1(=0.1) per attribute + Norm-in-Norm ranking loss + heteroscedastic NLL on .
- Face: CIoU box loss + focal (=0.25, =2) objectness + Wing loss landmarks + per-attribute BCE; skin-tone ITA regression with Huber.
- Lighting: MSE on SH coefficients + $(1 - \cos)$ angular loss on key-light direction + CE on scene-light categories.
- Segmentation: CE + Lovász-Softmax + boundary loss; hair class weighted $\times 3$ (hardest edges, highest user visibility).

Editing & color

- Edit Vector: MSE on normalized params, weighted by per-param confidence (learned); **through-renderer loss**: ΔE2000 + 0.5·LPIPS + 0.1·L1 on rendered vs. expert image.

- Curve legality: soft monotonicity penalty + total-variation smoothness on curves and on LUT basis outputs (prevents banding).
- Color consistency: histogram-matching loss in Lab; **skin-tone protection loss** penalizing predicted transforms that push detected skin hues outside the plausible skin locus in CIELAB (this one loss kills the "orange face" failure mode).
- LUT generator: code regression to SVD codes (MSE) + applied-ΔE against true LUTs + InfoNCE style contrastive (same-LUT images $\rightarrow$ nearby codes; different LUTs $\rightarrow$ apart) $\rightarrow$ structures the manifold.

Distillation & efficiency

- Feature: MSE + CKA on S3/S4 maps vs. DINOv2-L (projected); attention-transfer maps in S4.
- Logit: KL (T=4) on scene/IQA heads vs. ensemble teachers.
- Trunk: student matches MoE-teacher outputs (logit + feature) before pruning/QAT.
- Restoration auxiliary (train-only head, discarded at export): L1+SSIM denoise/deblur on HDR+/ SIDD — shapes low-level features the IQA head exploits.

Multi-task balancing: uncertainty weighting (Kendall) as base + PCGrad to de-conflict gradients + per-stage task temperature sampling. Expect sceneIQA synergy; watch segLUT conflict (mitigate: LUT head reads CLS only, minimal shared gradients).

---

13. Mobile Runtime Decision

The 2026 landscape changed underneath this question: **NNAPI is deprecated (Android 15+)** — do not build on it. Vendor delegates are the path now.

Runtime	Platform	Strengths	Weaknesses	Verdict
CoreML	iOS	Neural Engine + GPU + CPU auto-scheduling; FP16 & palettized INT8/INT4; best perf/watt on Apple silicon	Apple-only; some ops fall to CPU — audit with CoreML Instruments	**iOS primary**
LiteRT (TFLite) + QNN/Hexagon delegate	Android (Snapdragon)	NPU: up to 10× GPU / 100× CPU on 8-Elite-class chips; mature GPU delegate fallback; MediaPipe ecosystem	Vendor delegates vary by chipset	**Android primary (Snapdragon)**

Table 5 – continued

Runtime	Platform	Strengths	Weaknesses	Verdict
NCNN (Vulkan)	Android + iOS	Best single-API cross-vendor GPU path (Adreno/Mali/ Apple via MoltenVK); excellent INT8 kernels; tiny binary; no vendor lock	CPU fallback slower than vendor NPUs	**Android universal fallback** + low-end tier
ONNX Runtime Mobile (QNN EP)	Android + iOS	One .onnx from training to device; QNN EP covers Hexagon; CoreML EP on iOS	Larger binary; EP coverage gaps need testing	**Alternative single-format path** if team prefers one artifact
MNN / TNN / Paddle-Lite	Android + iOS	Solid Alibaba/ Tencent engines	No decisive edge over NCNN for this graph	Skip (redundant)
ExecuTorch	Both	PyTorch-native future	NPU delegate maturity still catching up for vision graphs as of mid-2026	Watchlist for v2

Final runtime matrix:

- **iOS:** CoreML package, FP16 compute, weight palettization (8-bit LUT-style quantization via coremltools) → Neural Engine. Minimum target A14 (iPhone 12).
- **Android tier-1 (Snapdragon 7-series+, 2023+):** LiteRT with QNN delegate (INT8 W / INT16 A where supported), GPU-delegate fallback.
- **Android tier-2 (MediaTek/Exynos/older):** NCNN Vulkan INT8; CPU path only as last resort.
- **One master graph:** PyTorch → ONNX (opset 17, static shapes) → {coremltools, ai-edge-litert, ncnn converter}. CI gate: per-device latency + ΔE numerical-parity tests vs. FP32 reference.

Rendering (LUT/curves/composite) never goes through the NN runtime — it's hand-written Metal (iOS) and Vulkan/GL (Android) shaders. Trilinear LUT sampling on a 12 MP image is ~2–4 ms on any 2022+ mobile GPU and zero neural overhead.

---

14. Deployment: Conversion, Quantization, Loading

Quantization scheme (per-module, not one-size):

Module	Weights	Activations	Why
Trunk S1–S3 (conv)	INT8 per-channel	INT8 per-tensor	NPU sweet spot; RepConv folds BN cleanly
Trunk S3–S4 (attention)	INT8 per-channel	INT8 (Softmax-free lite attention avoids the INT8-softmax cliff)	Linear attention quantizes far better than softmax
Scene / IQA / Lighting heads	INT8	INT8	Robust regressions/ classifications
Edit-Vector Decoder outputs	INT8	**FP16 final layer**	Slider values feed pixels — keep ΔE drift < 0.5
LUT generator output	INT8	**FP16 output head**	Color-critical; banding lives here
Basis LUT store	INT8	n/a	Smooth tables quantize losslessly
Mask decoder	INT8	INT8	Masks tolerate noise; guided filter fixes edges

Process: cross-layer equalization (AIMET) → per-channel weight quantization → **QAT with LSQ for the final 10% of training** → per-platform calibration (500-image set spanning scenes × devices × skin tones). Numerical parity gate: rendered-output ΔE00 < 1.0 vs. FP32 on a 1,000-image golden set.

Weight compression & packaging:

- Model ships as 3 packages: `lumen-core` (trunk+heads, ~95 MB), `lumen-lut` (basis+generator, ~6 MB), `lumen-air` optional W4A16 palettized build (~55 MB) for low-end SKUs.
- App-store delivery: on-demand module download (Play Feature Delivery / iOS On-Demand Resources) so the base APK/IPA stays lean.

Memory & loading:

- `mmap` weights (both CoreML and LiteRT/NCNN support zero-copy mapping) — no full copy into RAM; pages loaded on demand, evictable under pressure.
- **Lazy loading:** mask decoder and LUT generator load on first use (auto-enhance doesn't always need masks); scene/IQA/lighting heads are always resident (2 MB combined).
- **Arena allocator:** one activation arena sized once at init (~120 MB at 384^2 FP16), reused across all heads; in-place activations; no per-call allocations.
- **Cache:** LRU of last 8 computed adaptive LUTs keyed by (image-hash, style-code) — slider re-adjusts after the first are nearly free. Edit Vector JSON cached per image so re-opening a photo is 0 ms of NN compute.
- Peak RAM target: **350 MB** on a 6 GB device (weights resident ~110 + arena ~120 + image buffers ~80 + framework ~40).

Generated by Kimi.ai

15. Runtime Optimization Strategy

1. **Graph surgery pre-export:** fold BN, fuse RepConv branches, fuse SE-lite into adjacent GEMMs where the delegate supports it, convert global-average-pools to reduce ops.
2. **Static everything:** fixed 384^2 input, fixed token counts, fixed merge ratios, fixed mask-slot count. Mobile NPUs reward boring graphs with 2–5× over dynamic ones.
3. **Two-resolution trick:** scene/IQA run every edit; the Edit Decoder runs at most once per (image, intent) — user slider tweaks after that are pure shader re-renders (0 NN cost).
4. **Conditional head execution** (§4.4): scene-gated. Measured on a 50K-image distribution: average 35% of head compute skipped.
5. **Thermal/battery policy:** burst budget — first edit full-speed; sustained batch edits drop to FP16-GPU path or 320^2 analysis to stay under thermal envelope; never run trunk at max clock back-to-back for >2 s.
6. **Pipelining:** trunk on NPU full-res guided-filter prep on GPU LUT mix on CPU — the three stages overlap for batch edits.
7. **Delegate audit in CI:** every op must land on NPU/GPU; an op falling to CPU fails the build (CoreML Instruments / LiteRT profiler).

---

16. Model Size Breakdown (Target: 100–120 MB)

Component	Params	Precision	Size
Stem + S1 + S2 (RepConv CNN)	13.5M	INT8	13.5 MB
S3 hybrid (4 conv + 4 LiteMSA)	21M	INT8	21.0 MB
S4 transformer (+ToMe, CLS/registers)	18M	INT8	18.0 MB
Lite-FPN neck	5.5M	INT8	5.5 MB
Scene head	0.5M	INT8	0.5 MB
IQA head	1.5M	INT8	1.5 MB
Face head (det + attr)	6M	INT8	6.0 MB
Lighting head	1.5M	INT8	1.5 MB
Edit-Vector Decoder (4L×256d, 64 queries)	9M	INT8 (+FP16 out)	9.2 MB
Neural LUT generator (style encoder + mixer)	5M	INT8 (+FP16 out)	5.2 MB
Basis LUT store ($64 \times 17^3 \times 3$)	0.94M values	INT8	0.9 MB
Mask decoder (12-class) + guided-filter refiner	9.5M	INT8	9.5 MB
Embeddings, codebooks, task-router, metadata	—	mixed	2.0 MB
LUMEN-110 total	**~92M params**		**98.5 MB**
Headroom (v1.1 features, safety margin)			~15 MB

Optional `lumen-air` (W4A16 palettized trunk + pruned 10%): ~**55 MB** for budget devices, ~1.5% quality cost.

---

17. Estimated Inference Speed

Full auto-enhance pipeline (analyze → Edit Vector → adaptive LUT → masks → full-res 12 MP apply), warm start:

Device class	Example SoC	Path	Trunk	Heads (avg, conditional)	LUT gen	Full-res apply	**Total**
Flagship 2025-26	Snapdragon 8 Elite Gen 5	NPU INT8	9–14 ms	5–10 ms	1–2 ms	8–12 ms	**25–40 m** ✅
Flagship iOS	A18/A19 Pro	ANE FP16	10–16 ms	5–10 ms	1–2 ms	6–10 ms	**25–40 m** ✅
Flagship 2023	SD 8 Gen 2 / A16	NPU/ANE	15–22 ms	8–14 ms	2 ms	10–14 ms	**35–55 m** ✅
Mid-range 2024-25	SD 7s Gen 3 / Dimensity 7300	GPU/ Vulkan INT8	45–70 ms	20–35 ms	4 ms	14–20 ms	**85–130** ✅
Mid-range 2022	SD 778G / Dimensity 1080	GPU INT8	70–110 ms	30–50 ms	6 ms	18–26 ms	**125–190** ⚠️ (meet spec with 320^2 anal mode: ~ 150 ms)
Low-end (lumen-air)	Helio G99 class	CPU/GPU	–	–	–	–	**250–400** (acceptab for stills)

<150 ms on flagships: **met with 3–5× margin**, which is what enables live preview scrubbing later.

---

18. One Model vs. a Model Zoo

Dimension	Separate-model stack	LUMEN-110
Assets on device	Face det+attr (~40 MB) + seg (~60–200 MB) + IQA (~30 MB) + scene (~20 MB) + enhancer (~150–300 MB) + 5,000 LUT files (~1 GB) **1.3–1.6 GB**	~**99 MB** (13–16× smaller)
Feature extraction	6–8 separate backbone passes	**1 pass**
Latency (flagship, full pipeline)	350–800 ms cumulative	**25–55 ms**
Peak RAM	700 MB–1.2 GB	350 MB

Generated by Kimi.ai

Table 9 – continued

Dimension	Separate-model stack	LUMEN-110
Cross-task consistency	None (seg disagrees with face model, etc.)	Shared representation → masks, faces, edits agree by construction
Maintainability	6–8 runtimes/conversions/ calibrations	1 graph, 1 calibration
Extensibility	Add model #9	Add queries/heads to one trunk
Per-task SOTA ceiling	Slightly higher per niche (a 300 MB enhancer wins raw PSNR)	~1–3% behind per-task giants, invisible in ΔE terms
Battery per edit	High (repeated feature extraction)	~60–70% lower

The honest trade: a task-dedicated giant can edge out LUMEN on its one benchmark. You are buying a 15× smaller, 10× faster, coherent system at ~1–2% per-task quality — the right trade for a shipping product, and the gap shrinks every distillation cycle.

---

19. Technology Stack

Layer	Choice
Training	PyTorch 2.x + Lightning; timm/OpenCLIP for inits; FSDP for MoE-teacher phase
Data engine	**Rust** (Polars/Arrow, `image`, `zune-jpeg`) for edit-triplet/LUT-pair/degradation generation at 10 scale; WebDataset shards for training
Differentiable renderer	Custom PyTorch module (curves as monotone splines, HSL, 3D-LUT trilinear, mask compositing) —must bit-match the shipping shaders
Quantization	AIMET (equalization + QAT orchestration), LSQ; coremltools palettization for iOS
Export	ONNX opset 17 → coremltools / ai-edge-litert / ncnn
On-device NN	CoreML (iOS) ·LiteRT+QNN / NCNN Vulkan (Android)
On-device render	Metal shaders (iOS) ·Vulkan/GL compute (Android); shared GLSL-ish source, per-platform ports
App	Swift/SwiftUI (iOS), Kotlin/Compose (Android); C++ core for render engine shared via NDK
Eval harness	Golden-set ΔE parity, per-device latency CI, MOS-correlation (SRCC/PLCC) for IQA, human side-by-side panel for aesthetics
Privacy telemetry	Opt-in param-delta feedback only, never images

---

20. Consolidated Feature List

Scene (28): portrait, landscape, food, wildlife, pets, macro, architecture, street, indoor, outdoor, travel, night, sunset, sunrise, HDR, snow, rain, beach, forest, product, document, selfie, studio, low-light, backlit, sports, wedding, event. **Quality (14):** blur, motion blur, noise, dynamic range, sharpness, WB error, color cast, exposure error, JPEG artifacts, lens distortion, chromatic aberration, haze, fog, low contrast. **Face (12+):** quality, skin tone, skin exposure, eye visibility, face brightness, orientation, smile, glasses, beard, hair region, makeup, multi-face. **Lighting (9):** direction, intensity, shadow strength, highlight clipping, ambient, indoor, golden hour, blue hour, artificial. **Edit Vector:** 8 tone + 8 presence + 4 color + 32 curve + 24 HSL + 7 grading + 7 effects + 20 local-mask + 5 face + 65 style/LUT + confidences = 256-float structured output, fully user-adjustable. **LUT intelligence:** 5,000 LUTs → 64-basis manifold (~170× compression), image-adaptive generation, style search/interpolation, intensity control, .cube export. **Masks (12):** sky, body, face-skin, hair, clothes, background, building, water, vegetation, road, vehicle, animal — coarse + guided full-res.

---

21. Research Papers to Study (by module)

Mobile backbones: RepViT (CVPR'24) ·FastViT (ICCV'23) ·EfficientViT (CVPR'23) ·EfficientFormerV2 (ICCV'23) ·MobileNetV4 ('24) ·EdgeNeXt ·MobileViTv3. **Token/efficiency:** Token Merging/ToMe (ICLR'23) ·DynamicViT ·A-ViT ·PiToMe ·ToFu. **MoE (train-time):** Switch Transformer · Soft MoE ·"MoE-ification" distillation works. **LUT & enhancement:** *Learning Image-Adaptive 3D LUTs* (Zeng, CVPR'20 — the cornerstone) ·AdaInt (CVPR'22) ·SepLUT (ECCV'22) ·SA-3DLUT ('21) ·Bilateral-grid spatial-aware LUT (ECCV'24) ·HDRNet (TOG'17 — the original "analyze small, apply big") ·CSRNet ·4D LUT ('23). **Retouching/ params:** Deep Photo Enhancer ('18) ·UPE (SIGGRAPH'18) ·StarEnhancer ('21) ·PieNet ·DiffRetouch ('23) ·CLIPtone ('23) ·NeuralPreset ('23) · **VeraRetouch ('26 — latent + differentiable renderer + preference RL; closest to LUMEN's edit head)** ·SAC slider-agent works on FiveK. **Restoration teachers:** Restormer ·NAFNet ·Retinexformer ·Zero-DCE++. **IQA:** MUSIQ ·LIQE ·HyperIQA ·MANIQA ·Norm-in-Norm loss ·SPAQ paper. **Face/masks:** SCRFD ·RetinaFace ·MediaPipe Face Mesh ·MODNet ·CelebAMask-HQ. **Segmentation:** SegFormer ·DDRNet ·PP-LiteSeg ·SAM + MobileSAM/EdgeSAM (pseudo-labeling + distillation donors). **Self-supervision/ distillation:** DINOv2 ·MobileCLIP ·SigLIP ·TinyViT distillation ·Hinton KD · FitNets/attention transfer. **Multi-task/optimization:** Kendall uncertainty weighting ·GradNorm · PCGrad ·ASL (asymmetric loss). **Quantization:** LSQ/LSQ+ ·AIMET white papers (DFQ, cross-layer equalization, AdaRound) ·coremltools palettization docs. **Data:** MIT-Adobe FiveK ·PPR10K ·HDR+ (Hasinoff'16) ·RAW unprocessing (Brooks'19) ·SIDD.

22. Existing Models That Inspire Each Module

LUMEN module	Lineage
Trunk S1–S2	RepViT-M2, FastViT-SA (RepConv, MetaFormer)
Trunk S3–S4	EfficientViT (lite MSA), EfficientFormerV2 (latency-driven NAS mindset)
Token efficiency	ToMe (fixed-rate variant for static export)
Neck	Semantic-FPN, BiFPN-lite
Edit-Vector Decoder	DETR queries × HDRNet' s bilateral philosophy × VeraRetouch' s latent-through-renderer

Generated by Kimi.ai

Table 11 – continued

LUMEN module	Lineage
LUT system	Zeng image-adaptive 3D LUT + AdaInt + SepLUT + NeuralPreset' s style conditioning
IQA head	MUSIQ/LIQE (native-res, quality-scene joint) distilled into a trunk readout
Face head	SCRFD anchor-free design, attributes à la CelebAMask-HQ parsers
Mask decoder	SegFormer-B0 decoder + MODNet' s guided matting idea (distilled to 2-layer guided filter)
Lighting head	SH-lighting estimation from portrait-relighting literature (StyleLight lineage)
Train-time teachers	DINOv2-L (semantics), SigLIP-L (aesthetics), Restormer (low-level), SAM-H (masks)

23. Risks & Trade-offs (and mitigations)

Risk	Severity	Mitigation
Multi-task negative transfer (heads fight over trunk)	High	Uncertainty weighting + PCGrad; per-head LoRA-style adapters if conflict persists; monitor per-task Δ vs. single-task baselines in every experiment
Quantization color shift (INT8 on color paths $\rightarrow$ banding/cast)	High	FP16 on LUT/edit outputs; ΔE00 < 1.0 golden-set CI gate; QAT last 10% of training
FiveK taste is 2011-era	High	Commission a modern edit corpus (§11); preference-tune on current aesthetics; user-delta flywheel
LUT manifold can' t cover 5,000 styles with 64 bases	Medium	Cluster-then-basis (8×16); verify held-out reconstruction ΔE; allow 12 "hero LUTs" shipped verbatim (12×216 KB $= 2.6$ MB) if any fail reconstruction
Mask edge quality at /8 (hair halos)	Medium	Guided-filter upsampling + learned refiner; hair loss $\times 3$; matting-style alpha supervision on CelebAMask-HQ
Face dataset licensing/privacy (CelebA-HQ, LAION faces)	High	Legal review per set; prefer WIDER FACE + consented commissioned data; never ship face recognition, only editing attributes

Table 12 – continued

Risk	Severity	Mitigation
Android fragmentation (delegate coverage)	High	3-tier runtime matrix (§13); per-device CI farm (top-20 SoCs); NCNN Vulkan as the floor
Thermal throttling in batch editing	Medium	Burst budget policy, 320^2 fallback mode, pipeline overlap (§15)
Conditional-execution bugs (head skipped when needed)	Medium	Router thresholds tuned for high recall (face prior 0.1); "run all heads" debug mode; telemetry on skip rates
MoE→dense distillation gap	Low-Med	Budget 2 distillation cycles; keep MoE teacher for future re-distills
Scope creep (video, generative fill)	Medium	Roadmap §24 —v1 is stills only

---

24. Future Improvements (v1.1 → v2.0)

1. **On-device personalization:** per-user LoRA (2 MB) fine-tuned *on the device* from the user's slider overrides of AI suggestions — the editor learns *your* taste privately. This is the retention moat.
2. **Video mode:** temporal-consistency via per-clip LUT finetuning (10–20 iterations on keyframes, freeze LUT for the clip — proven approach in LUT stylization literature); needs the render engine only, no new trunk.
3. **W4A16 everywhere** → 55 MB default build as NPU INT4 support matures.
4. **RAW-domain processing** path for ProRAW/DNG (trunk already RAW-agnostic via unprocessing training).
5. **Generative add-on** (optional 400 MB download): small latent-diffusion module for object removal/generative fill — kept *separate* so the core stays 100 MB.
6. **NPU-native sparse MoE** in v2 if vendor delegates add dynamic-expert support — revisit the rejected deployment-MoE.
7. **Multi-frame fusion** (burst → one Edit Vector) using HDR+-style merging statistics.
8. **Accessibility & pro surfaces:** voice-driven edits ("warmer, less orange") via a tiny on-device intent parser mapping text → Edit Vector deltas.

---

25. The One-Paragraph Version

LUMEN-110 is a 92M-parameter hybrid CNN-transformer (RepConv stages → lite-attention stages → ToMe-accelerated global stage) running once at 384^2 INT8, feeding a DETR-style Edit-Vector Decoder that emits 256 structured, user-adjustable editing parameters; a neural-LUT module that compresses your 5,000-LUT library into a 64-basis manifold (~1 GB → ~6 MB) with image-adaptive generation and style interpolation; and a coarse 12-class mask decoder refined by guided filtering — trained by MoE-amplified multi-teacher distillation on FiveK/PPR10K/SPAQ + millions of synthetic edit triplets generated by

your own LUT collection, deployed as CoreML on iOS and LiteRT-QNN/NCNN on Android, landing at **~99 MB, 25–55 ms flagship latency, 350 MB RAM** — one model replacing a 1.5 GB model zoo.

Generated by Kimi.ai